

The Engineering Guide to LLM Inference, vLLM & KV Cache Systems

From Silicon Physics and Memory Paging to Production Deployment & Interview Mastery

Target Model: Qwen 2.5 0.5B Instruct	Hardware: NVIDIA Tesla T4 (16 GB VRAM)	Runtime: vLLM 0.29 + PagedAttention
Status: Production Verified	Context Capacity: 32,768 Tokens	Cost per Session: ~\$0.25 (35 mins)

1. Live Operational Benchmarks & Measured Numbers

During live deployment testing on our cloud Tesla T4 GPU, the following empirical performance metrics were captured:

Metric	Measured Value	Engineering Significance
Decode Throughput	76.83 tok/sec	Generation speed per user stream (~10x faster than human reading speed of ~8 tok/s).
Prefill Throughput	1,893.7 tok/sec	Compute-bound prompt ingestion rate over parallel Tensor Cores.
Model Weights (VRAM)	0.93 GiB	Qwen 2.5 0.5B in FP16 precision takes less than 1 GB of physical VRAM.
Allocated KV Cache	6.99 GiB	GPU memory partitioned strictly for dynamic token paging and attention state.
KV Cache Token Pool	590,512 tokens	Total concurrent token capacity before preemption or swapping is required.
Max Concurrency	576 requests	Simultaneous users supported at 1,024 context tokens without memory exhaustion.
Context Window	32,768 tokens	Scaled native context window providing 25,000+ free tokens of agent headroom.

2. The End-to-End Story: 16 Core Terminologies in Action

To understand modern LLM systems, we trace a single real-world prompt: *'Write a Python function to sort a list.'* Every concept is anchored directly to its place in execution.

Act I: The Static Setup

- Model Weights / Parameters:** The 500 million numeric weights (FP16 = 2 bytes per weight = ~1 GB) that represent the learned grammar and logic transferred over the PCIe bus into GPU High Bandwidth Memory (HBM).
- Precision & Quantization:** Storing weights in FP16 (2 bytes) vs. compressed formats like AWQ or GPTQ (INT4 = 0.5 bytes). Quantization shrinks a 70B model from 140 GB down to ~35 GB.

- **Tensor Parallelism (TP):** Slicing matrix multiplication operations across multiple GPUs in parallel. A 70B model with TP=4 splits every weight matrix across 4 GPUs with collective communication.

Act II: Input Tokenization & Chat Templating

- **Tokens & Tokenization:** The text string is sliced by byte-pair encoding into discrete integer IDs: [14251, 264, 9822, 2184, 311, 4920, 264, 1541].
- **Context Window:** The total allowable token buffer (prompt + generation). Scaled to 32,768 tokens on our server to ensure massive agent prompts never exceed hardware bounds.
- **Chat Template & Hermes Tool Parser:** Wraps raw user text with role tags (<|im_start|>user...<|im_end|>). The Hermes parser catches XML <tool_call> tokens and wraps them into standard OpenAI JSON schemas.

Act III: The Prefill Phase (Prompt Processing)

- **Prefill Phase:** All prompt tokens are processed simultaneously in parallel. This phase is **Compute-Bound** (\$GEMM\$), maximizing Tensor Core arithmetic utilization.
- **Time to First Token (TTFT):** The latency from request arrival until the first generated token appears on screen. Dominated by prefill computation and kernel compilation.
- **Attention (Q, K, V):** Query represents what a token seeks; Key represents its context; Value represents its meaning. $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$.
- **KV Cache:** Storing computed \$K\$ and \$V\$ tensors in GPU memory to prevent quadratic recalculation of previous tokens on every subsequent generation step.

Act IV: Memory Virtualization (PagedAttention)

- **PagedAttention:** Allocating KV memory in non-contiguous 16-token physical blocks mapped by an OS-style page table, reducing memory waste from 80% to under 3%.
- **Prefix Caching (Radix Tree):** Indexing token sequences so repeated system prompts and tool schemas are computed once and reused with \$O(1)\$ lookup latency.
- **CUDA Graphs:** Recording execution graph kernels to eliminate host CPU launch overhead on fixed tensor batch sizes.

Act V: The Decode Phase (Token Generation)

- **Decode Phase:** Generating tokens autoregressively one-by-one. This phase is **Memory-Bandwidth-Bound** (\$GEMV\$), constrained by the rate at which weights can be read from VRAM.
- **Inter-Token Latency (ITL) & TPS:** The delay between subsequent streaming words. Measured at 76.83 tokens/second (~13 ms per token).
- **Continuous Batching (In-Flight):** Dynamically inserting new incoming requests and retiring completed sequences at every single token iteration without pipeline stalls.

3. Deep-Dive Answers to Core Architectural Questions

Q1: What is an OpenAI-Compatible API vs. a Regular REST API?

A regular REST API uses arbitrary endpoints (e.g., POST /generate) and proprietary JSON schemas. The OpenAI API is the industry-standard specification (/v1/chat/completions, messages array, choices array). Exposing an OpenAI-compatible endpoint allows tools like Cursor, Continue, LangChain, and Open WebUI to connect with zero code changes simply by changing the base_url.

Q2: Why Did Tool Calling Fail with Error 400?

When an agent passes "tool_choice": "auto", vLLM expects an explicit tool-call parser. LLMs are text-only sequence predictors; Qwen outputs tool calls in raw XML (<tool_call>{"name": "..."}</tool_call>). Without --tool-call-parser hermes, vLLM cannot extract the function call into the structured JSON expected by the agent, resulting in a 400 Bad Request.

Q3: What Does 'Breaking Memory into Chunks' Mean (PagedAttention)?

Traditional systems pre-allocated huge contiguous blocks of GPU memory (e.g., 2,048 tokens in a straight row) in case a user typed a long prompt. If a user only typed 10 tokens, 2,038 slots were frozen and wasted (up to 80% internal/external fragmentation). PagedAttention breaks memory into 16-token pages scattered anywhere in physical VRAM, tracked by an OS-like Page Table.

Q4: What Happens to Unused Slots in a 16-Token Page on Short Prompts?

When a user sends a 3-token prompt like 'Hey', slots 4 through 16 are not wasted. As the model generates its response ('Hello! How can I help...'), it immediately writes the new tokens into the remaining slots of that exact page. Only when token 17 is reached is a new page allocated. Internal fragmentation is restricted to at most 15 tokens on the final tail page (< 3% waste).

Q5: How Does NVIDIA TensorRT-LLM Compare?

TensorRT-LLM adopted the exact same paged KV cache concept. However, while vLLM is Python-first and dynamically compiles models in seconds, TensorRT-LLM is C++-based and requires ahead-of-time engine compilation (trtllm-build). TRT-LLM also uses larger block sizes (64 or 128 tokens) to optimize for hardware memory bus bursts (128-byte cache lines) on Hopper/Ada silicon.

Q6: Why Did the First Request Take 22 Seconds?

The initial prompt contained 6,145 tokens (system prompt, tools, workspace tree). Because this was a new tensor shape, PyTorch Triton paused to perform a one-time JIT compilation of the attention kernel (WARNING: Triton kernel JIT compilation during inference). Once compiled and cached, subsequent requests responded in milliseconds.

Q7: Why Did Context Overflow and How Did 32K Fix It?

The agent prompt (6,145 tokens) plus requested output (2,048 tokens) totaled 8,193 tokens—exceeding the server limit by exactly 1 token. By upgrading vLLM to --max-model-len 32768 (supported natively by Qwen 2.5), the prompt consumed only ~18% of context, leaving 25,000+ free tokens of headroom.

4. Essential AI Systems Interview Questions & Model Answers

Interview Scenario 1: Back-of-the-Napkin VRAM Sizing

Question: 'Can we serve Llama 3 70B with batch size 8 and 4,096 context in FP16 on two 80 GB A100s?'

Model Answer: Weights require $70 \times 2 = 140 \text{ GB}$. KV cache for GQA (80 layers, 8 heads, 128 dim) requires $2 \times 80 \times 8 \times 128 \times 2 = 327.68 \text{ KB/token}$. For 4,096 tokens across batch size 8: $4096 \times 8 \times 0.328 \text{ MB} \approx 10.75 \text{ GB}$. Total memory = $140 + 10.75 + 5 \text{ (activations)} = 155.75 \text{ GB}$. While mathematically under 160 GB, 97% utilization will trigger OOM under traffic spikes; Tensor Parallelism across 4 GPUs or FP8 quantization is strongly recommended.

Interview Scenario 2: Prefill vs. Decode Hardware Bottlenecks

Question: 'Why is prefill compute-bound while decode is memory-bandwidth-bound?'

Model Answer: In prefill, thousands of tokens are processed simultaneously in a large matrix-matrix multiplication (\$GEMM\$), reusing loaded weights across all tokens and saturating Tensor Cores. In decode, tokens are generated one-by-one (\$GEMV\$). For every single token, the GPU must read all model weights from HBM to compute just one vector dot product. The compute units sit idle 90% of the time waiting on memory bus transfers. Continuous batching raises arithmetic intensity by batching decode steps across multiple concurrent users.

Interview Scenario 3: Memory Fragmentation & Prefix Caching

Question: 'How does PagedAttention eliminate external fragmentation, and how does prefix caching integrate with it?'

Model Answer: Traditional allocators require contiguous memory, failing when fragmented gaps exist. PagedAttention allocates non-contiguous 16-token physical blocks indexed by a Page Table. Prefix caching indexes these blocks by token content hashes in a Radix Tree. Matching prompt prefixes point to already-resident physical blocks, eliminating redundant prefill computation.

5. Production Deployment Scripts & Reference Commands

```
# Launch vLLM with 32K context and Hermes tool calling
vllm serve Qwen/Qwen2.5-0.5B-Instruct \
--host 0.0.0.0 \
--port 8000 \
--gpu-memory-utilization 0.75 \
--max-model-len 32768 \
--enable-auto-tool-choice \
--tool-call-parser hermes
```